

ReneWind Project

Post Graduate Program in Artificial Intelligence and Machine Learning

Prepared by Christian Caceres

March 15, 2026

Contents / Agenda

- Executive Summary
- Business Problem Overview and Solution Approach
- Data Overview
- EDA Results
- Data Preprocessing
- Model Evaluation Approach
- Final Model Selection
- Test Performance of Best Model
- Conclusions and Recommendations

Executive Summary

ReneWind aims to improve wind turbine maintenance by developing a machine learning classification model that predicts generator failures using historical operational and sensor data. Early detection of potential failures enables the company to perform preventive maintenance, reducing costly generator replacements and minimizing operational downtime.

Six neural network models were developed with increasing architectural complexity. **Model 0**, the baseline model, achieved a recall of 0.90, showing strong ability to detect generator failures. **Model 1** added an additional hidden layer and maintained the same recall performance. **Model 2** introduced dropout and a deeper architecture to reduce overfitting, though recall slightly decreased to 0.88. **Models 3–5** further improved the architecture by incorporating class weights to address class imbalance and experimenting with the Adam optimizer, resulting in stable training behavior and good generalization between training and validation loss. **Model 6**, which used tanh activation with SGD and class weights, performed significantly worse with a recall of 0.43, indicating many missed failures and clear underfitting, making it the weakest model among those tested.

After evaluating multiple models, **model 3** was selected due to its strong performance, achieving **95.6% recall** and an **95.6% F1-score**. Its high recall directly aligns with the business objective to identify generator failures before they occur.

The proposed solution will help **ReneWind** by enabling early detection of potential generator failures, allowing the company to take preventive action before a breakdown occurs..

Business Problem Overview and Solution Approach

Problem Overview:

- **RenewWind** is a company that focuses on improving the reliability and efficiency of wind energy production by using machine learning techniques. Wind turbines rely on generators and other mechanical components that can fail over time due to continuous operation and environmental conditions. Unexpected generator failures can lead to significant maintenance costs, downtime, and loss of energy production.
- To address this issue, **RenewWind** has collected sensor data from wind turbine generators that monitor different operating conditions. The goal of this project is to develop classification models that can predict potential generator failures before they occur. Early detection allows maintenance teams to perform repairs in advance, preventing major breakdowns and reducing overall maintenance costs.

Solution Approach:

- To address the problem, **RenewWind** will develop a machine learning classification model that predicts whether a wind turbine generator is likely to fail based on historical operational and sensor data.
- The approach includes applying regularization techniques such as dropout, using class weights to address class imbalance, experimenting with different optimizers like SGD and Adam, and increasing the number of neural network layers to improve model performance.
- The best-performing model will be selected using evaluation metrics such as recall, precision, and F1-score, with particular emphasis on recall due to the high cost associated with undetected failures.
- The final solution will provide actionable insights into the key drivers of generator failures and enable proactive maintenance planning, helping reduce unexpected breakdowns, lower replacement costs, and improve overall operational efficiency.

Data Overview

Train Data Sample

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	...	V32	V33
0	-4.464606	-4.679129	3.101546	0.506130	-0.221083	-2.032511	-2.910870	0.050714	-1.522351	3.761892	...	3.059700	-1.690440
1	3.365912	3.653381	0.909671	-1.367528	0.332016	2.358938	0.732600	-4.332135	0.565695	-0.101080	...	-1.795474	3.032780
2	-3.831843	-5.824444	0.634031	-2.418815	-1.773827	1.016824	-2.098941	-3.173204	-2.081860	5.392621	...	-0.257101	0.803550
3	1.618098	1.888342	7.046143	-1.147285	0.083080	-1.529780	0.207309	-2.493629	0.344926	2.118578	...	-3.584425	-2.577474
4	-0.111440	3.872488	-3.758361	-2.982897	3.792714	0.544960	0.205433	4.848994	-1.854920	-6.220023	...	8.265896	6.629213

Test Data Sample

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	...	V32	V33
0	-0.613489	-3.819640	2.202302	1.300420	-1.184929	-4.495964	-1.835817	4.722989	1.206140	-0.341909	...	2.291204	-5.411388
1	0.389608	-0.512341	0.527053	-2.576776	-1.016766	2.235112	-0.441301	-4.405744	-0.332869	1.966794	...	-2.474936	2.493582
2	-0.874861	-0.640632	4.084202	-1.590454	0.525855	-1.957592	-0.695367	1.347309	-1.732348	0.466500	...	-1.318888	-2.997464
3	0.238384	1.458607	4.014528	2.534478	1.196987	-3.117330	-0.924035	0.269493	1.322436	0.702345	...	3.517918	-3.074085
4	5.828225	2.768260	-1.234530	2.809264	-1.641648	-1.406698	0.568643	0.965043	1.918379	-2.774855	...	1.773841	-1.501573

The dataset contains multiple numerical features labeled V1–V40, which appear to be continuous variables.

The values include both positive and negative numbers, suggesting that the variables may have already been scaled or standardized, which is common in datasets prepared for machine learning models.

Overall, both datasets share the same feature format and numerical structure, which is necessary for reliable model training and testing. This consistency ensures that the trained model can be properly applied to the test data for prediction and performance evaluation.

Data Overview

Statistical Summary

	count	mean	std	min	25%	50%	75%	max
V1	19982	-0.272	3.44163	-11.876	-2.7371	-0.7479	1.84011	15.493
V2	19982	0.44043	3.15078	-12.32	-1.6407	0.47154	2.54397	13.0893
V3	20000	2.4847	3.38896	-10.708	0.20686	2.25579	4.56617	17.0909
V4	20000	-0.0832	3.4316	-15.082	-2.3477	-0.1352	2.13062	13.2364
V5	20000	-0.0538	2.1048	-8.6034	-1.5356	-0.102	1.34048	8.1338
V6	20000	-0.9954	2.04097	-10.227	-2.3472	-1.0005	0.38033	6.97585
V7	20000	-0.8793	1.76163	-7.9497	-2.0309	-0.9172	0.2237	8.00609
V8	20000	-0.5482	3.29576	-15.658	-2.6427	-0.3891	1.72297	11.6795
V9	20000	-0.0168	2.16057	-8.5963	-1.495	-0.0676	1.4092	8.13758
V10	20000	-0.013	2.1932	-9.854	-1.4112	0.10097	1.47705	8.10847
V11	20000	-1.8954	3.12432	-14.832	-3.9224	-1.9212	0.11891	11.8264
V12	20000	1.60483	2.93045	-12.948	-0.3965	1.50784	3.57145	15.0807
V13	20000	1.58049	2.87466	-13.228	-0.2235	1.63719	3.45989	15.4196
V14	20000	-0.9506	1.78965	-7.7386	-2.1707	-0.9572	0.27068	5.67066
V15	20000	-2.415	3.35497	-16.417	-4.4153	-2.3826	-0.3591	12.2465
V16	20000	-2.9252	4.22172	-20.374	-5.6342	-2.6827	-0.095	13.5832
V17	20000	-0.1343	3.34546	-14.091	-2.2156	-0.0146	2.06875	16.7564
V18	20000	1.18935	2.59228	-11.644	-0.4039	0.8834	2.57177	13.1799
V19	20000	1.18181	3.39693	-13.492	-1.0502	1.27906	3.4933	13.2377
V20	20000	0.02361	3.66948	-13.923	-2.433	0.03342	2.51237	16.0523
V21	20000	-3.6113	3.56769	-17.956	-5.9304	-3.5329	-1.2659	13.8405
V22	20000	0.95184	1.65155	-10.122	-0.1181	0.97469	2.02559	7.40986
V23	20000	-0.3661	4.03186	-14.866	-3.0988	-0.2621	2.45175	14.4587
V24	20000	1.13439	3.91207	-16.387	-1.4681	0.96905	3.54598	17.1633
V25	20000	-0.0022	2.01674	-8.2283	-1.3652	0.02505	1.39711	8.22339
V26	20000	1.87379	3.43514	-11.834	-0.3379	1.95053	4.13004	16.8364
V27	20000	-0.6124	4.36885	-14.905	-3.6523	-0.8849	2.18918	17.5604
V28	20000	-0.8832	1.91771	-9.2695	-2.1712	-0.8911	0.37588	6.52764
V29	20000	-0.9856	2.68437	-12.579	-2.7874	-1.1762	0.62977	10.7221
V30	20000	-0.0155	3.00526	-14.796	-1.8671	0.18435	2.03623	12.5058
V31	20000	0.48684	3.46138	-13.723	-1.8178	0.4903	2.73069	17.2551
V32	20000	0.3038	5.5004	-19.877	-3.4205	0.05207	3.76172	23.6332
V33	20000	0.04983	3.57529	-16.898	-2.2429	-0.0662	2.25513	16.6925
V34	20000	-0.4627	3.18384	-17.985	-2.137	-0.255	1.43694	14.3582
V35	20000	2.22962	2.9371	-15.35	0.33619	2.09863	4.06436	15.2911
V36	20000	1.51481	3.80086	-14.833	-0.9438	1.56653	3.98394	19.3296
V37	20000	0.01132	1.78817	-5.4784	-1.2558	-0.1284	1.17553	7.46701
V38	20000	-0.344	3.94815	-17.375	-2.9876	-0.3168	2.2794	15.2899
V39	20000	0.89065	1.75305	-6.4389	-0.2723	0.91926	2.05754	7.75988
V40	20000	-0.8756	3.01216	-11.024	-2.9402	-0.9208	1.1199	10.6543
Target	20000	0.0555	0.22896	0	0	0	0	1

- Total observations: 20,000
- Total features: 40 predictor variables + 1 target variable
- V1 and V2 have 19,982 observations, meaning 18 missing values each – missing values should be imputed
- Several variables show large ranges and potential outliers

Because variables have different ranges and variances, scaling is recommended before modeling

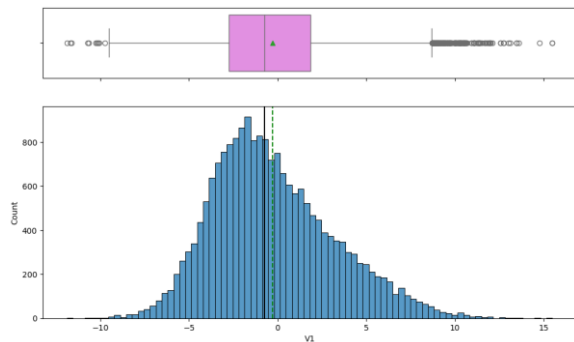
Data Overview

Neural Network Data Compatibility Transformation V40 (Target Variable) **int64** → **float64**

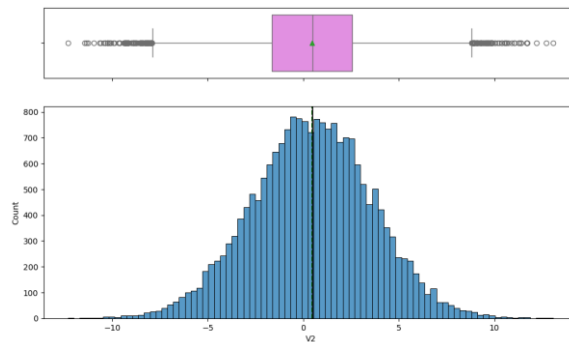
Train Data	Convert Target to Float	Neural Network
<pre> 32 V33 20000 non-null float64 33 V34 20000 non-null float64 34 V35 20000 non-null float64 35 V36 20000 non-null float64 36 V37 20000 non-null float64 37 V38 20000 non-null float64 38 V39 20000 non-null float64 39 V40 20000 non-null float64 40 Target 20000 non-null int64 dtypes: float64(40), int64(1) memory usage: 6.3 MB </pre>	<pre>data['Target'] = data['Target'].astype(float)</pre>	Ensures numerical compatibility with machine learning algorithms, especially neural networks.
Test Data	Convert Target to Float	Neural Network
<pre> 32 V33 5000 non-null float64 33 V34 5000 non-null float64 34 V35 5000 non-null float64 35 V36 5000 non-null float64 36 V37 5000 non-null float64 37 V38 5000 non-null float64 38 V39 5000 non-null float64 39 V40 5000 non-null float64 40 Target 5000 non-null int64 dtypes: float64(40), int64(1) memory usage: 1.6 MB </pre>	<pre>data_test['Target'] = data_test['Target'].astype(float)</pre>	Ensures numerical compatibility with machine learning algorithms, especially neural networks.

EDA Results: V1-V5

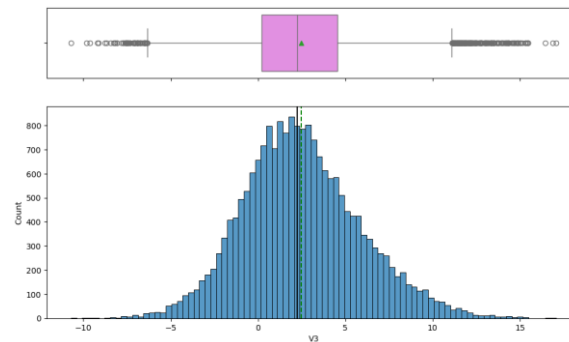
V1



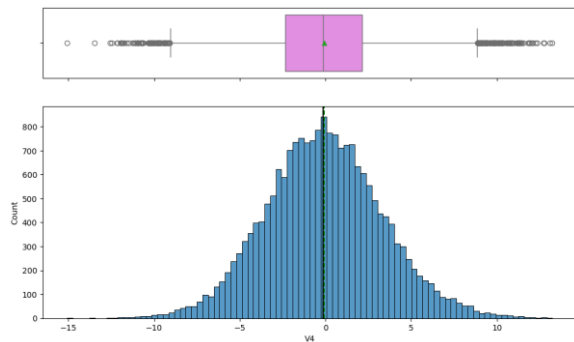
V2



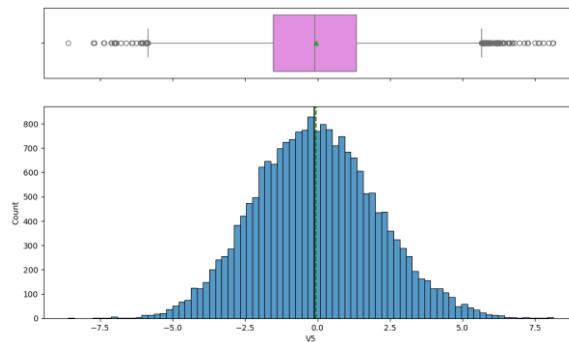
V3



V4



V5



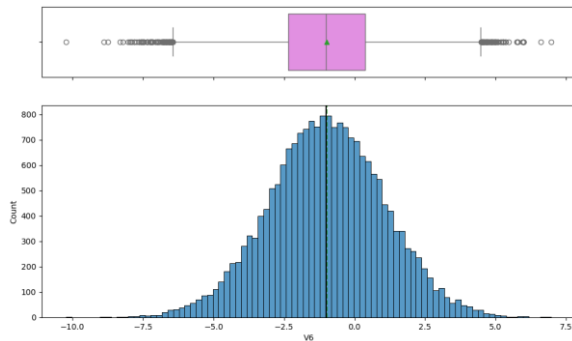
V1 and V3 show moderately wide spreads, indicating noticeable variability in their values. Both variables display slight right skewness, with longer positive tails visible in their histograms.

V2, V4, and V5 exhibit approximately symmetric bell-shaped distributions centered close to zero. **V4** shows a wider spread, suggesting higher variability, while **V5** has the smallest spread, indicating more concentrated values around the mean.

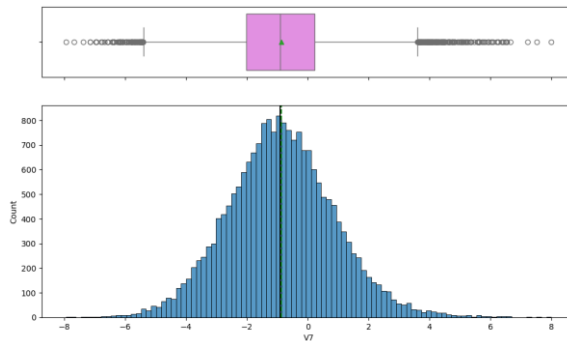
Outliers are present across all variables

EDA Results: V6-V10

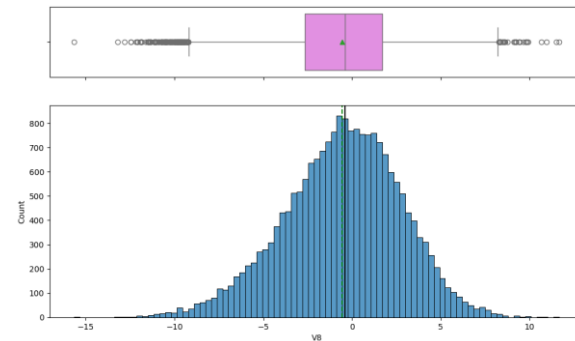
V6



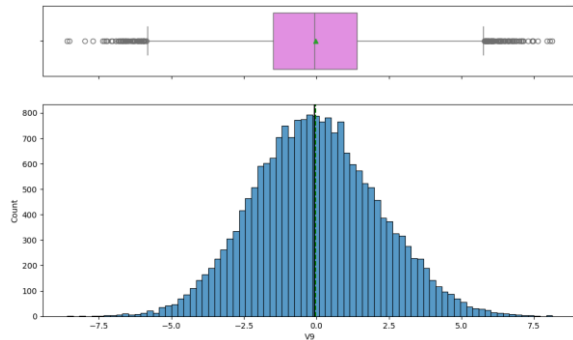
V7



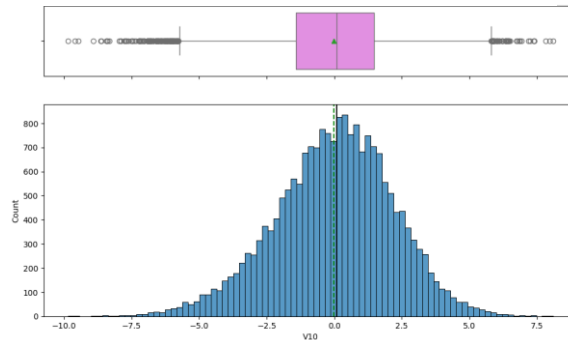
V8



V9



V10



V6 and V9 show moderate spreads with values concentrated around the center, indicating relatively stable variability.

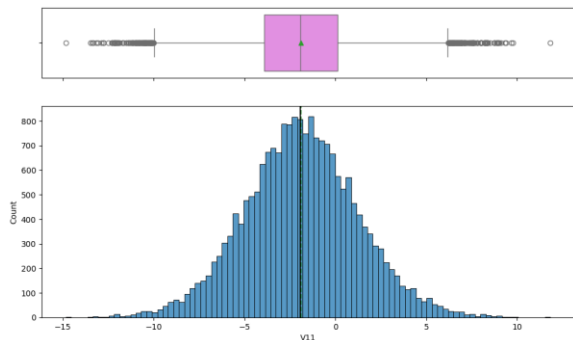
V7 and V10 display slightly wider distributions, suggesting somewhat greater variability compared to the other variables.

V8 shows the widest spread among the variables, indicating higher variability and a broader range of values.

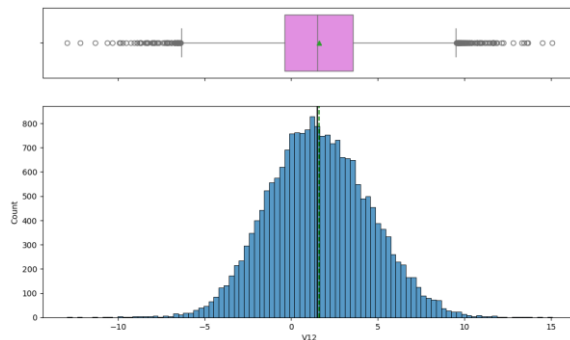
Outliers are present across all variables

EDA Results: V11-V15

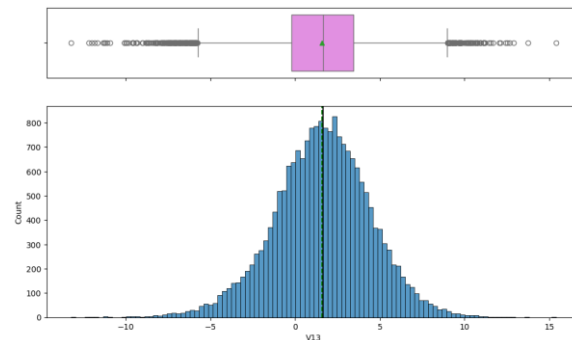
V11



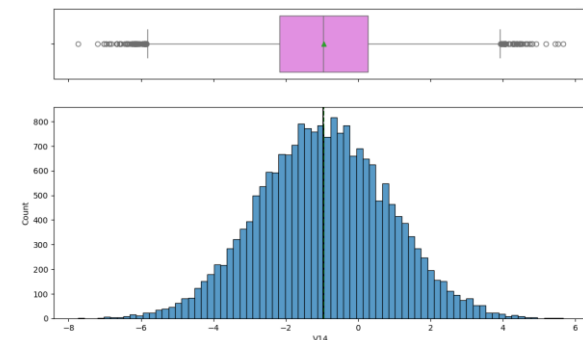
V12



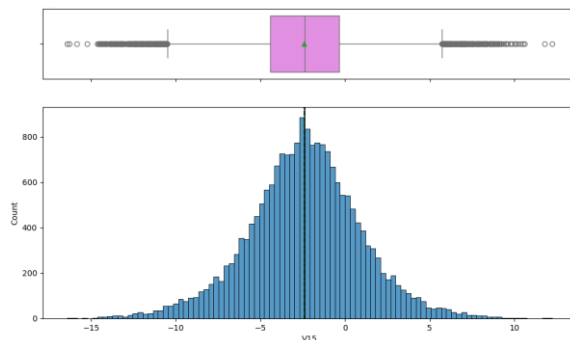
V13



V14



V15



V11 and V15 show wider spreads compared to the other variables, indicating higher variability and a broader range of values.

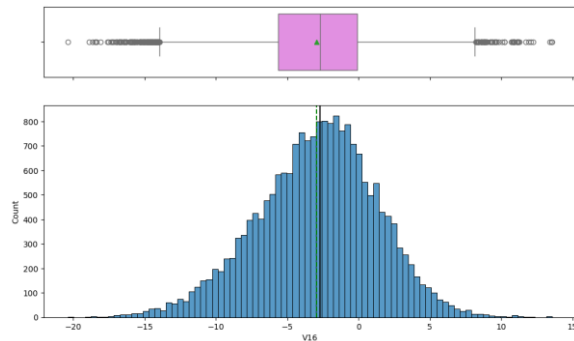
V12 and V13 exhibit approximately symmetric bell-shaped distributions centered around values slightly above zero.

V14 shows a narrower spread, indicating lower variability and more tightly clustered values compared to the other variables.

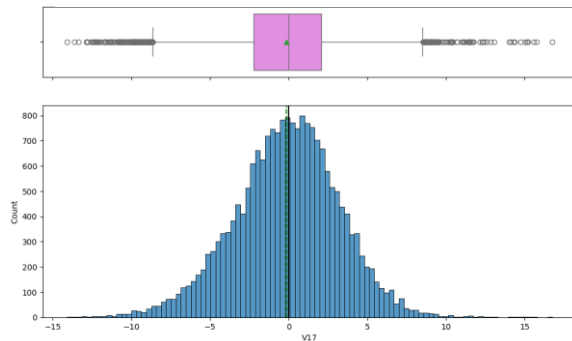
Outliers are present across all variables

EDA Results: V16-V20

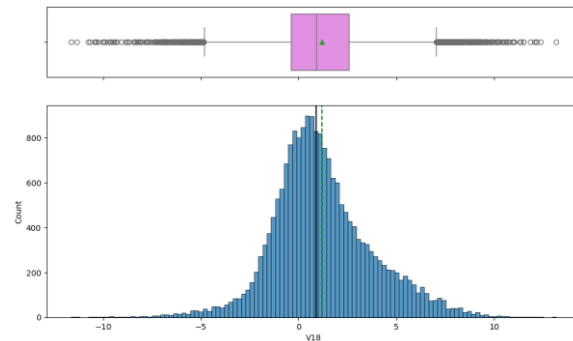
V16



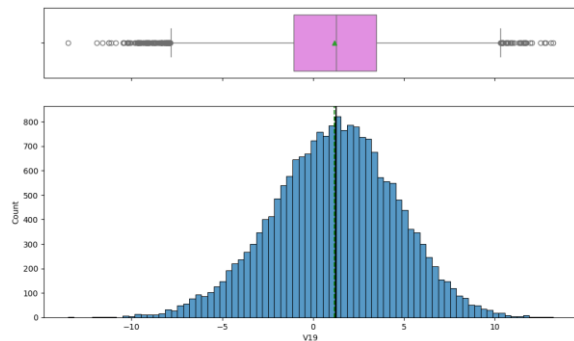
V17



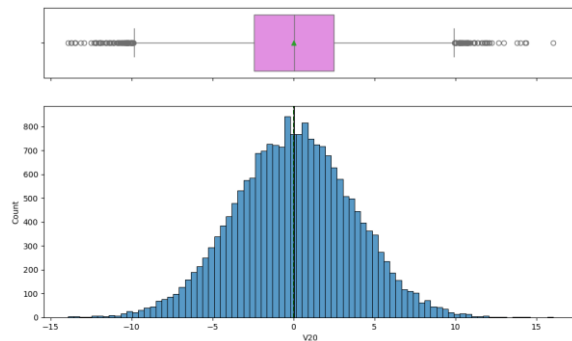
V18



V19



V20



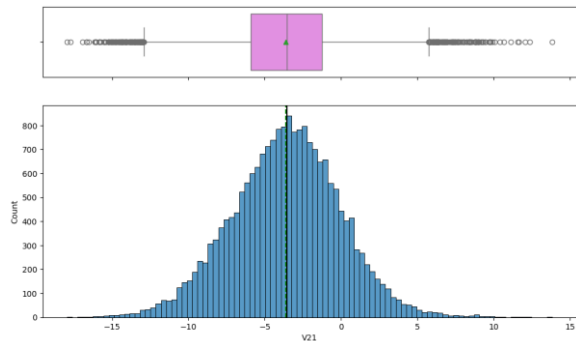
V16 and V18 show relatively wider spreads, indicating higher variability in their values. Both variables also display slightly longer tails in their histograms.

V17, V19, and V20 exhibit approximately symmetric bell-shaped distributions centered near zero. Their spreads appear moderate, with most observations concentrated around the mean.

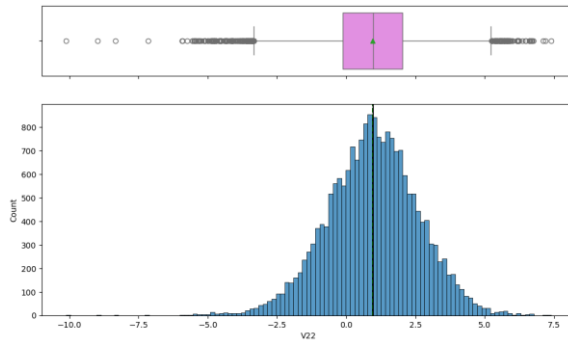
Outliers are present across all variables

EDA Results: V21-V25

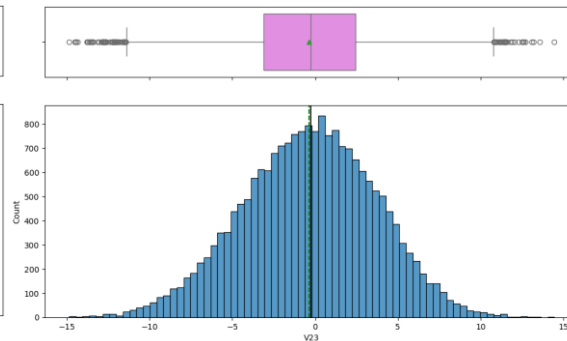
V21



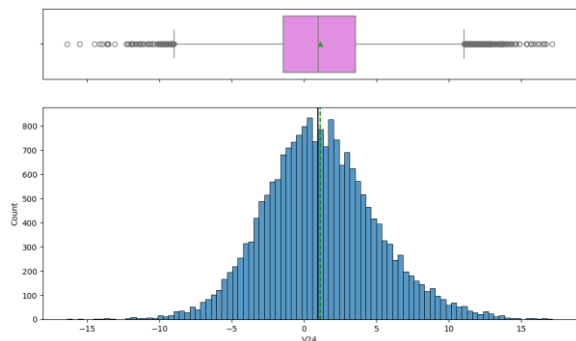
V22



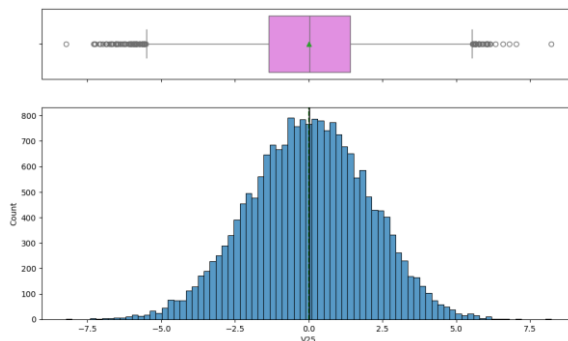
V23



V24



V25



V21 and V24 show wider spreads, indicating higher variability and a broader range of values compared to the other variables.

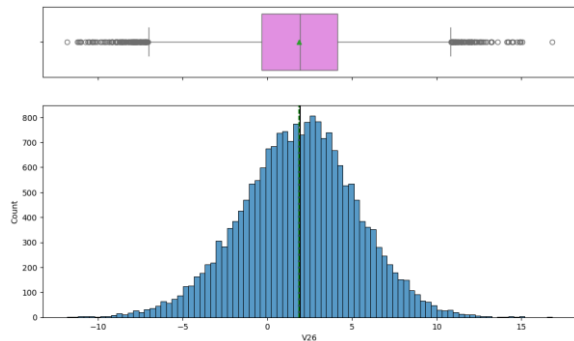
V22 and V25 display narrower distributions, with values more tightly concentrated around the mean. This indicates lower variability and more stable observations.

V23 shows a moderate spread, falling between the wider and narrower distributions, with most values centered around zero.

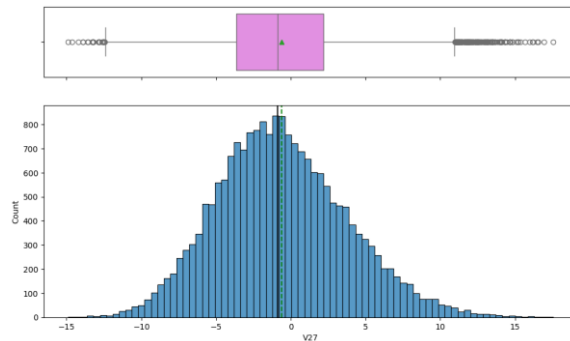
Outliers are present across all variables

EDA Results: V26-V30

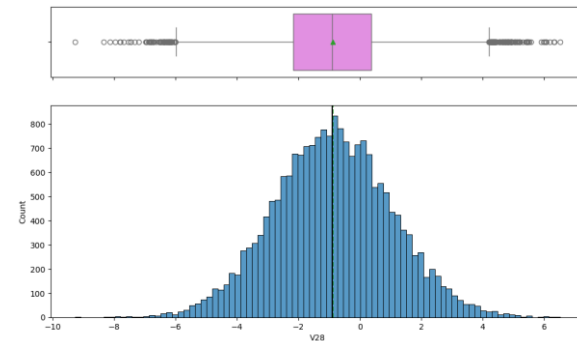
V26



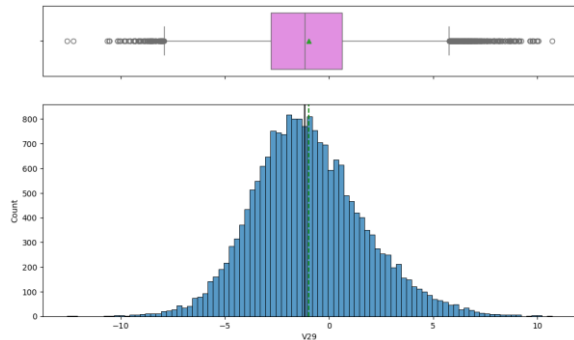
V27



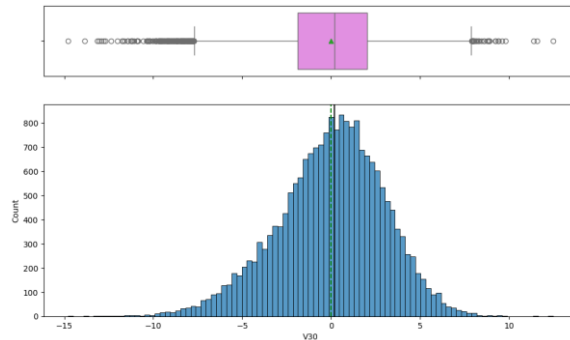
V28



V29



V30



V26 and V27 show wider spreads, indicating greater variability in their values.

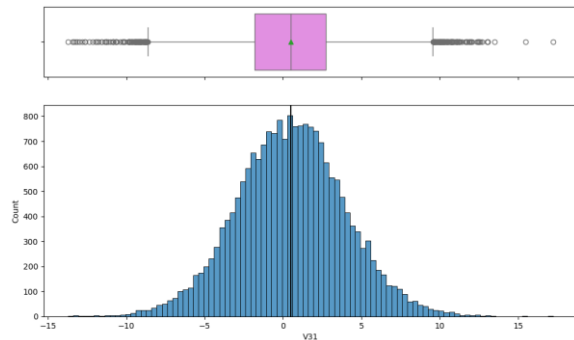
V29 and V30 display moderate spreads with approximately bell-shaped distributions centered close to zero.

V28 shows the narrowest spread among the variables, with values tightly clustered around the center, indicating lower variability compared to the others.

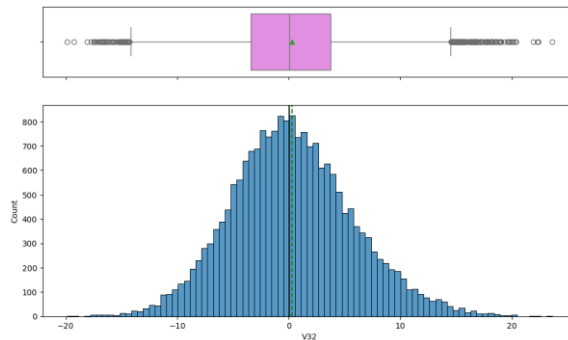
Outliers are present across all variables

EDA Results: V31-V35

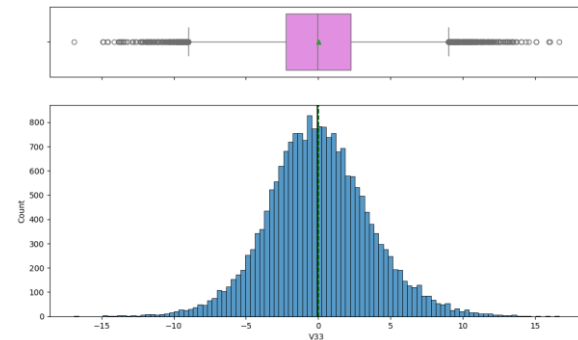
V31



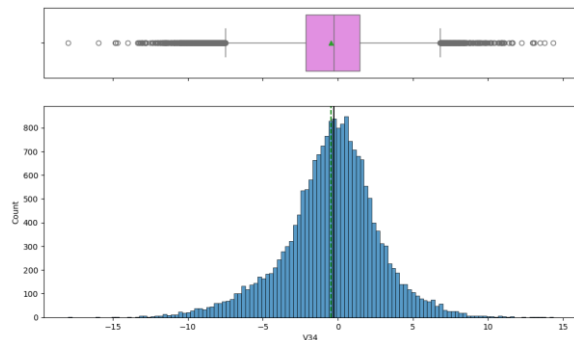
V32



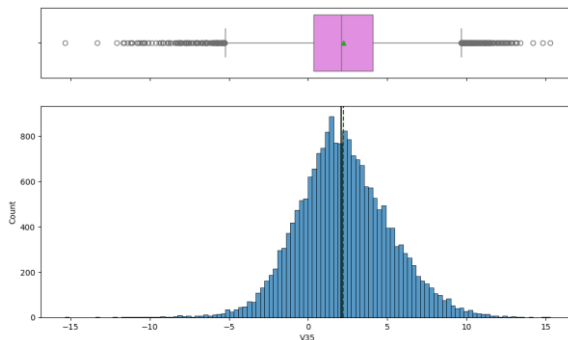
V33



V34



V35



V31 and V32 show relatively wider spreads compared to the other variables, indicating greater variability in their values.

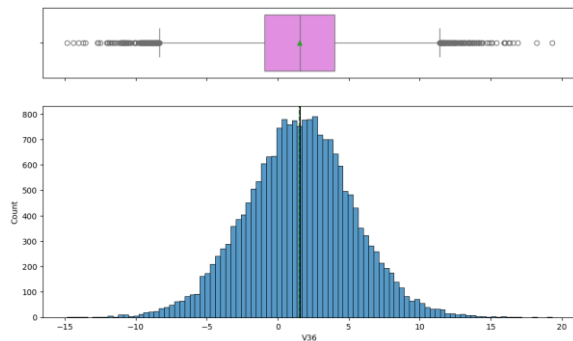
V33 and V35 exhibit moderate spreads with approximately bell-shaped distributions centered near zero.

V34 shows a slightly narrower spread, with values more tightly clustered around the center compared to the others.

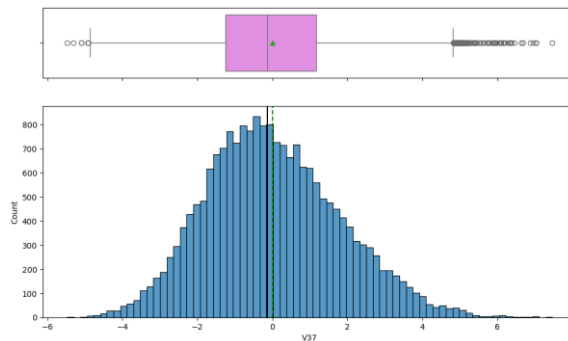
Outliers are present across all variables

EDA Results: V36-V40

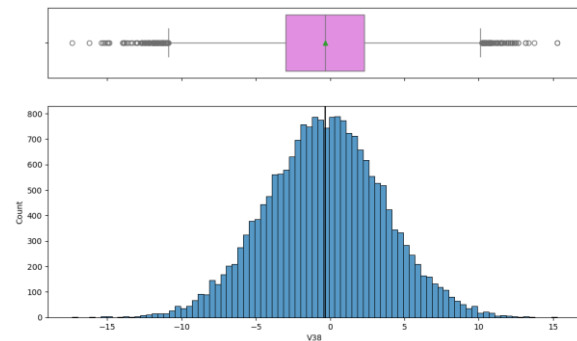
V36



V37

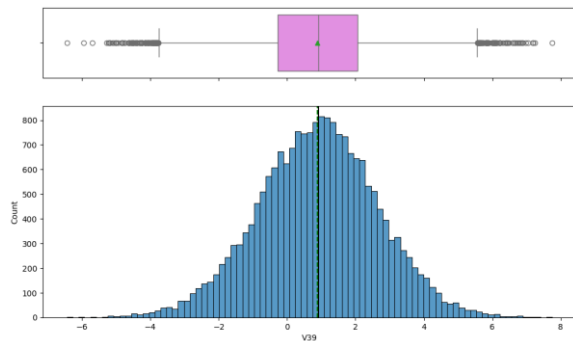


V38

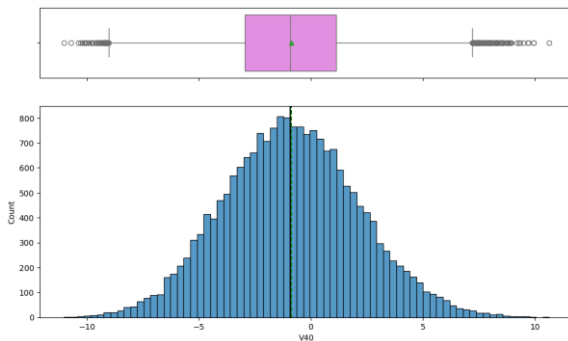


V36 and V38 show relatively wider spreads, indicating greater variability and a broader range of values.

V39



V40



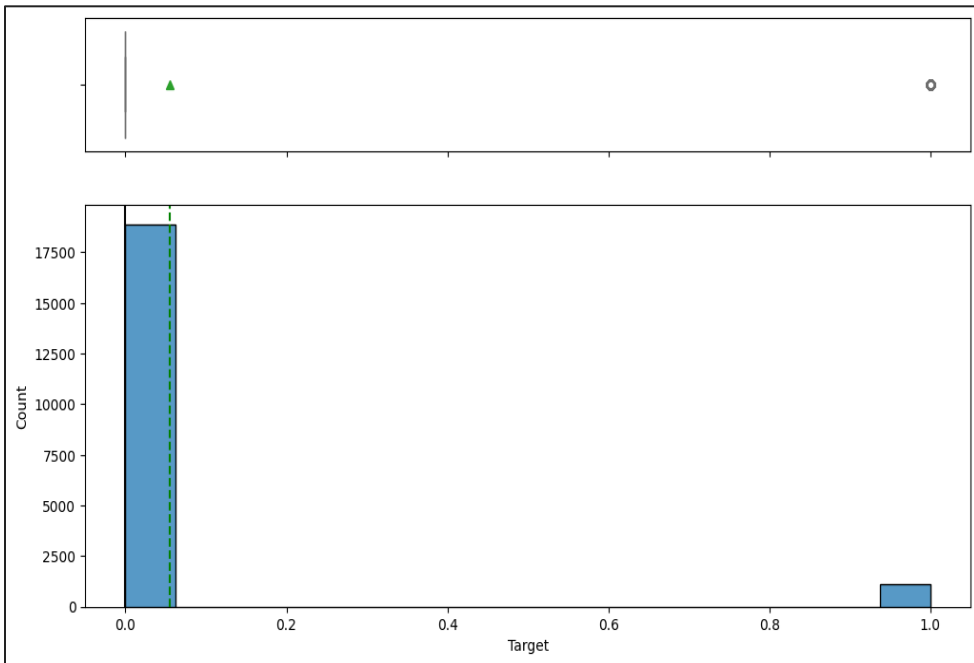
V37 and V40 display moderate spreads with approximately bell-shaped distributions centered close to zero.

V39 shows the narrowest spread among the variables, with values tightly clustered around the center, indicating lower variability compared to the others.

Outliers are present across all variables

EDA Results: Target Variable

Target Variable

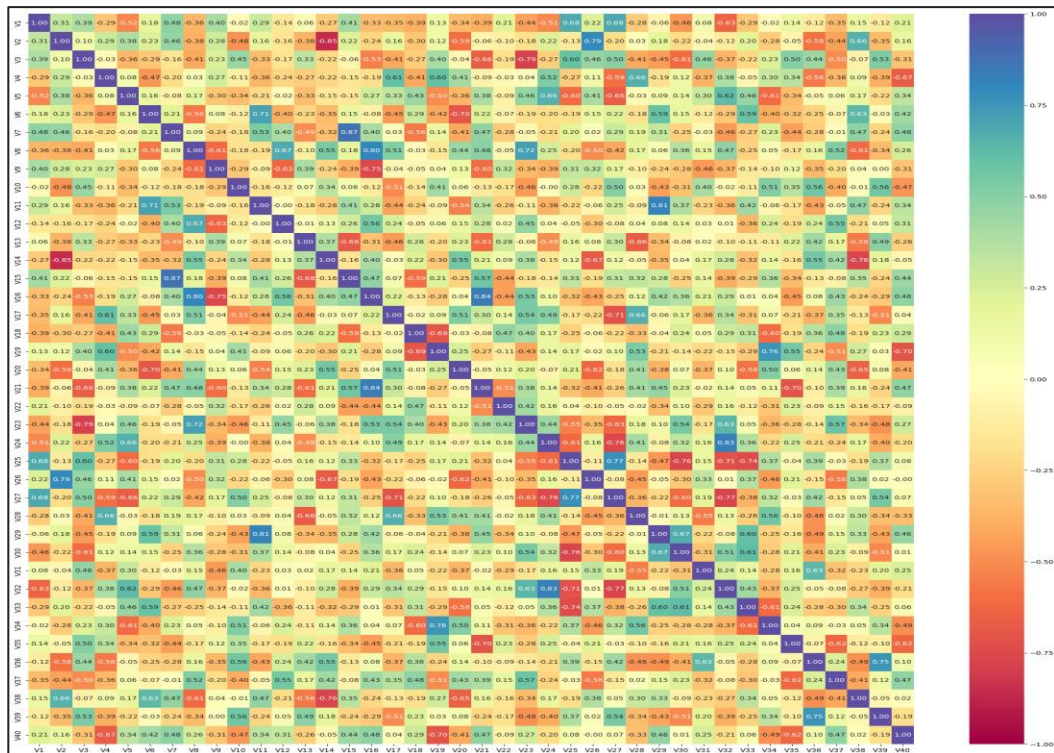


The Target variable shows a highly imbalanced distribution between the two classes. Most observations belong to class 0 while class 1 appears to be underrepresented.

Techniques such as class weighting should be used during the model building phase to handle the imbalance

EDA Results

Correlation Matrix: V1-V40



Most variable pairs show low to moderate correlation levels which make the data set suitable for Neural Network modeling

Overall, the correlation analysis indicates that the dataset has low overall multicollinearity, making the variables suitable for machine learning modeling without the need for major feature removal based on correlation.

Data Preprocessing

Duplicate Values

```
data.duplicated().sum()  
  
np.int64(0)
```

There are no duplicate values

Missing Values

Train

```
data.isnull().sum()  
...  
0  
V1 18  
V2 18  
V3 0
```

Test

```
data_test.isnull().sum() #  
0  
V1 5  
V2 6  
V3 0
```

Impute missing values for V1 and V2 (Train, Test and Validation) using median.

```
imputer = SimpleImputer(strategy="median")
```

- `X_train = pd.DataFrame(imputer.fit_transform(X_train), columns=X_train.columns)`
- `X_val = pd.DataFrame(imputer.transform(X_val), columns=X_train.columns)`
- `X_test = pd.DataFrame(imputer.transform(X_test), columns=X_train.columns)`

Data Preprocessing

Data Split: Test and Validation

```
▶ # Splitting data into training and validation set:  
  
X_train, X_val, y_train, y_val = train_test_split(  
    X, y, test_size=0.25, random_state=1, stratify=y  
)
```

The data set was split into 25% validation and 75% training

- ❑ Training Set: (15000, 40)
- ❑ Validation Set: (5000, 40)

Data Split: Test Set

```
# Dividing test data into X_test and y_test  
X_test = data_test.drop(columns = ['Target'])  
y_test = data_test["Target"] # Complete the c
```

Dividing test set into X_test and y_test

- ❑ Test Set: (5000, 40)

Model Evaluation Approach

Performance Metrics

Problem Statement

For the ReneWind project, the primary goal of the classification model is to identify generator failures before they occur so that preventive maintenance can be performed. Because the cost of missing an actual failure (False Negative) is significantly higher than the cost of unnecessary inspections (False Positives).

Primary Metric:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Recall is a model evaluation metric that measures how well a classification model identifies the actual positive cases.

In the ReneWind project, recall measures how effectively the model detects turbine generator failures

Secondary Metrics:

F1 and Precision

- F1 score helps evaluate the balance between detecting generator failures (recall) and avoiding unnecessary inspections (precision)
- Precision score helps control the number of false alarms which leads to reducing unnecessary inspections.

Neural Network: Model 0

Parameters

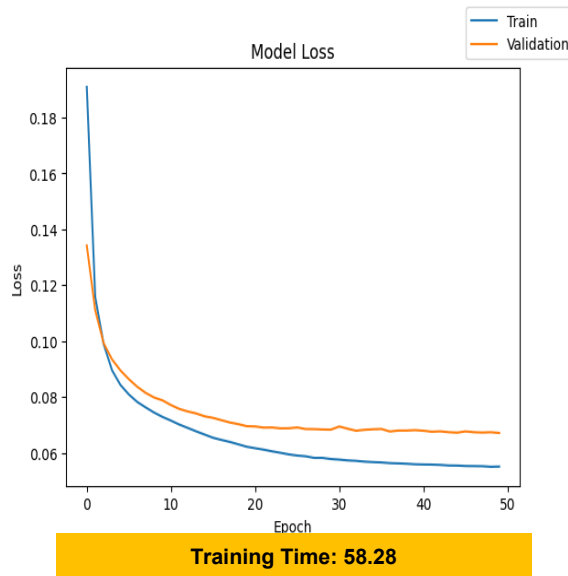
Training

- Epochs: 50
- Batch Size: 32
- Activation Function Hidden Layer (s): ReLU
- Activation Function Output Layer: Sigmoid
- Optimizer: SGD
- Total Params: 295
- Optimizer Metric: Recall

Validation

- Epochs: 50
- Batch Size: 32
- Activation Function Hidden Layer (s): ReLU
- Activation Function Output Layer: Sigmoid
- Optimizer: SGD
- Total Params: 295
- Optimizer Metric: Recall

Model Loss Graph



Performance Metrics

- Accuracy: 0.98
 - Recall: 0.90
 - Precision: 0.97
 - F1: 0.93
-
- Accuracy: 0.98
 - Recall: 0.90
 - Precision: 0.96
 - F1: 0.93

- **Model 0** is a baseline neural network trained with 50 epochs, batch size of 32, ReLU activation in the hidden layer, Sigmoid in the output layer, and SGD as the optimizer, with Recall as the optimization metric.
- The loss curves show a sharp decline in both training and validation loss during the first few epochs, followed by a gradual stabilization. The validation loss remains slightly higher than the training loss but follows a similar pattern, indicating that the model is learning effectively with no significant overfitting.
- The recall of 0.90 is particularly important because it means the model correctly identifies most generator failures, reducing the risk of undetected failures that could lead to expensive generator replacements

Neural Network: Model 1

Parameters

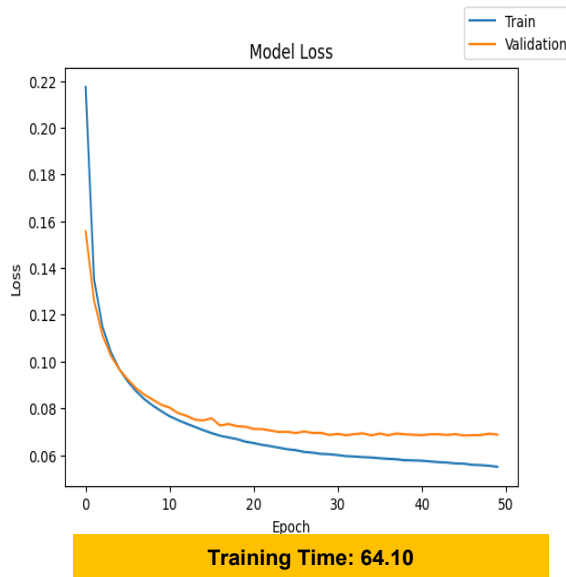
Training

- Epochs: 50
- Batch Size: 32
- Activation Function Hidden Layer 1: ReLU
- Activation Function Hidden Layer 2: ReLU
- Activation Function Output Layer: Sigmoid
- Optimizer: SGD
- Total Params: 414
- Optimizer Metric: Recall

Validation

- Epochs: 50
- Batch Size: 32
- Activation Function Hidden Layer 1: ReLU
- Activation Function Hidden Layer 2: ReLU
- Activation Function Output Layer: Sigmoid
- Optimizer: SGD
- Total Params: 414
- Optimizer Metric: Recall

Model Loss Graph



Performance Metrics

- Accuracy: 0.98
 - Recall: 0.90
 - Precision: 0.97
 - F1: 0.93
-
- Accuracy: 0.98
 - Recall: 0.90
 - Precision: 0.96
 - F1: 0.93

- **Model 1** expands on the baseline model by adding an additional hidden layer. The model is trained with 50 epochs, a batch size of 32, ReLU activation in both hidden layers, Sigmoid activation in the output layer, and SGD as the optimizer, while optimizing for recall. The total number of parameters increases to 414, making the model slightly more complex than Model 0.
- The loss graph shows that both training and validation loss decrease rapidly in the initial epochs and gradually stabilize as training progresses. The validation loss closely follows the training loss with only a small gap between them, indicating that the model generalizes well and does not exhibit significant overfitting.
- The recall of 0.90 is particularly important because it means the model correctly identifies most generator failures, reducing the risk of undetected failures that could lead to expensive generator replacements.

Neural Network: Model 2

Parameters

Model Loss Graph

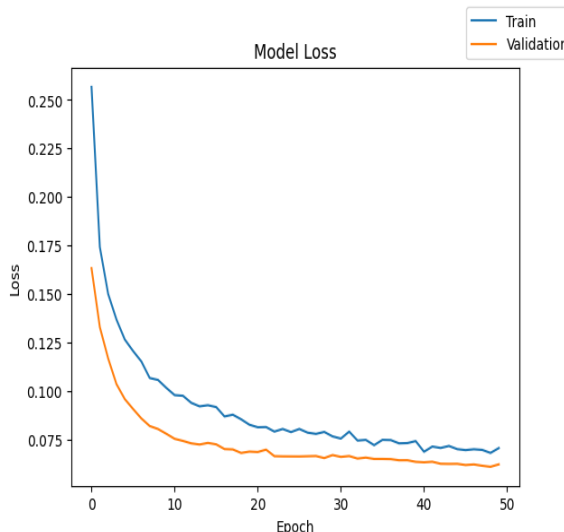
Performance Metrics

Training

- Epochs: 50
- Batch Size: 32
- Activation Function Hidden Layer 1: ReLU
- Dropout (.50)
- Activation Function Hidden Layer 2: ReLU
- Activation Function Hidden Layer 3: ReLU
- Activation Function Output Layer: Sigmoid
- Optimizer: SGD
- Total Params: 1985
- Optimizer Metric: Recall

Validation

- Epochs: 50
- Batch Size: 32
- Activation Function Hidden Layer 1: ReLU
- Dropout (.50)
- Activation Function Hidden Layer 2: ReLU
- Activation Function Hidden Layer 3: ReLU
- Activation Function Output Layer: Sigmoid
- Optimizer: SGD
- Total Params: 1985
- Optimizer Metric: Recall



Training Time: 65.18

- Accuracy: 0.98
 - Recall: 0.88
 - Precision: 0.98
 - F1: 0.92
-
- Accuracy: 0.98
 - Recall: 0.89
 - Precision: 0.98
 - F1: 0.93

- **Model 2** increases the complexity of the neural network by adding an additional hidden layer and introducing dropout (50%) after the first hidden layer to reduce overfitting. The model is trained for 50 epochs with a batch size of 32, uses ReLU activation in all hidden layers, Sigmoid activation in the output layer, and SGD as the optimizer, while optimizing for recall. Due to the deeper architecture, the model contains 1,985 parameters, which is significantly higher than the previous models.
- The loss graph shows that both training and validation loss decrease steadily across epochs. The validation loss remains slightly lower than the training loss throughout training, which is expected when dropout is used because dropout is applied during training but not during validation. This pattern suggests that the model generalizes well and that the regularization technique is helping control overfitting.
- Recall of .88 indicates a significant impact to failure detection resulting on missing more real failures when compared to model 0 and model 1

Neural Network: Model 3

Parameters

Model Loss Graph

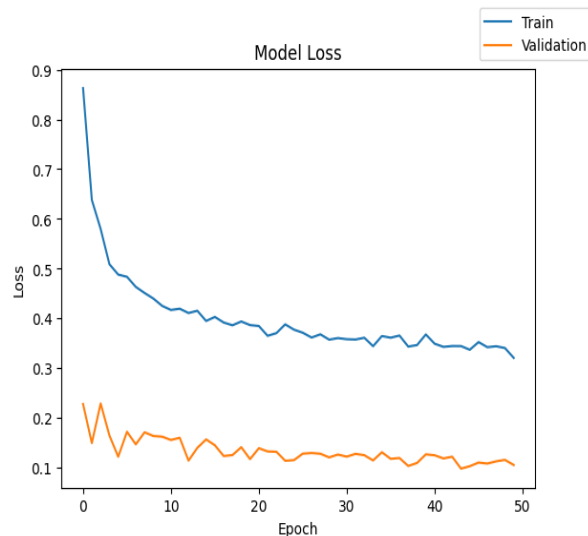
Performance Metrics

Training

- Epochs: 50
- Batch Size: 32
- Activation Function Hidden Layer 1: ReLU
- Dropout (.50)
- Class Weight: 0: (1.05) 1: (18.02)
- Activation Function Hidden Layer 2: ReLU
- Activation Function Hidden Layer 3: ReLU
- Activation Function Output Layer: Sigmoid
- Optimizer: SGD
- Total Params: 5249
- Optimizer Metric: Recall

Validation

- Epochs: 50
- Batch Size: 32
- Activation Function Hidden Layer 1: ReLU
- Dropout (.50)
- Class Weight: 0: (1.05) 1: (18.02)
- Activation Function Hidden Layer 2: ReLU
- Activation Function Hidden Layer 3: ReLU
- Activation Function Output Layer: Sigmoid
- Optimizer: SGD
- Total Params: 5249
- Optimizer Metric: Recall



Training Time: 65.99

- Accuracy: 0.99
 - Recall: 0.95
 - Precision: 0.95
 - F1: 0.95
-
- Accuracy: 0.99
 - Recall: 0.94
 - Precision: 0.97
 - F1: 0.95

- **Model 3** is a neural network trained with 50 epochs and a batch size of 32. The architecture includes three hidden layers using ReLU activation functions, followed by a Sigmoid activation in the output layer for binary classification. A dropout rate of 0.5 is applied to reduce overfitting, and class weights are used to address the strong class imbalance in the dataset. The model uses the SGD optimizer and is optimized primarily for recall.
- The loss curves show that training loss steadily decreases across epochs, indicating that the model is learning patterns from the training data. The validation loss remains consistently lower than the training loss, which can occur when dropout is applied during training but not during validation. Both curves stabilize toward the later epochs, suggesting the model converges well without clear signs of overfitting.

Neural Network: Model 4

Parameters

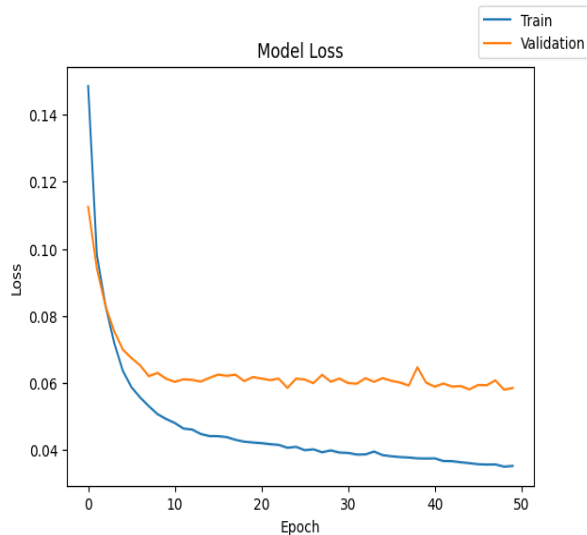
Training

- Epochs: 50
- Batch Size: 32
- Activation Function Hidden Layer 1: ReLU
- Activation Function Hidden Layer 2: ReLU
- Activation Function Output Layer: Sigmoid
- Optimizer: Adam
- Total Params: 414
- Optimizer Metric: Recall

Validation

- Epochs: 50
- Batch Size: 32
- Activation Function Hidden Layer 1: ReLU
- Activation Function Hidden Layer 2: ReLU
- Activation Function Output Layer: Sigmoid
- Optimizer: Adam
- Total Params: 414
- Optimizer Metric: Recall

Model Loss Graph



Training Time: 65.25

Performance Metrics

- Accuracy: 0.99
 - Recall: 0.95
 - Precision: 0.98
 - F1: 0.96
-
- Accuracy: 0.99
 - Recall: 0.93
 - Precision: 0.98
 - F1: 0.95

- **Model 4** uses a neural network with 50 epochs and a batch size of 32. The architecture consists of three hidden layers with ReLU activation functions, followed by a Sigmoid output layer for binary classification. A dropout rate of 0.5 is applied to reduce overfitting. Unlike Model 3, this model uses the Adam optimizer, which generally improves convergence speed and stability. The model is optimized using recall as the primary metric, which aligns with the objective of detecting generator failures.
- The training loss decreases rapidly during the early epochs and gradually stabilizes, indicating that the model is learning meaningful patterns from the data. The validation loss follows a similar pattern, remaining close to the training loss throughout the training process – we can observe good model convergence and very little overfitting

Neural Network: Model 5

Parameters

Model Loss Graph

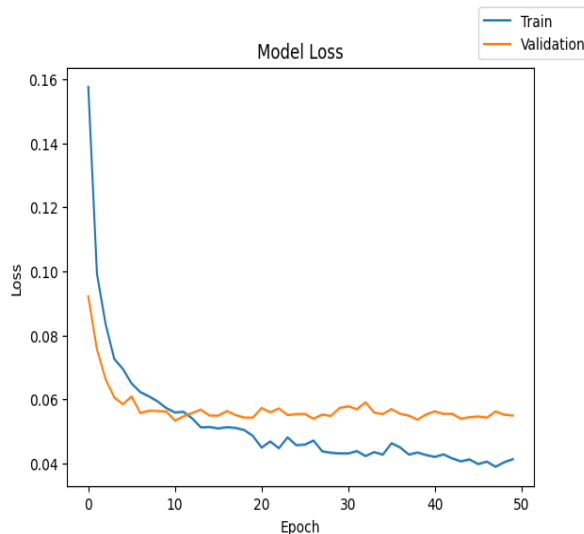
Performance Metrics

Training

- Epochs: 50
- Batch Size: 32
- Activation Function Hidden Layer 1: ReLU
- Dropout (.50)
- Activation Function Hidden Layer 2: ReLU
- Activation Function Hidden Layer 3: ReLU
- Activation Function Output Layer: Sigmoid
- Optimizer: Adam
- Total Params: 5249
- Optimizer Metric: Recall

Validation

- Epochs: 50
- Batch Size: 32
- Activation Function Hidden Layer 1: ReLU
- Dropout (.50)
- Activation Function Hidden Layer 2: ReLU
- Activation Function Hidden Layer 3: ReLU
- Activation Function Output Layer: Sigmoid
- Optimizer: Adam
- Total Params: 5249
- Optimizer Metric: Recall



Training Time: 70.64

- Accuracy: 0.99
 - Recall: 0.94
 - Precision: 0.99
 - F1: 0.96
-
- Accuracy: 0.99
 - Recall: 0.92
 - Precision: 0.98
 - F1: 0.95

- **Model 5** is trained using 50 epochs and a batch size of 32. The neural network consists of three hidden layers with ReLU activation functions and a Sigmoid activation in the output layer for binary classification. A dropout rate of 0.5 is applied to help reduce overfitting. The model uses the Adam optimizer, which provides faster and more stable convergence compared to SGD. The model is optimized with recall as the primary metric, which aligns with the objective of detecting generator failures
- The training loss decreases rapidly during the first few epochs and gradually stabilizes, indicating that the model is learning effectively. The validation loss remains close to the training loss throughout the training process, suggesting that the model is generalizing well without significant overfitting

Neural Network: Model 6

Parameters

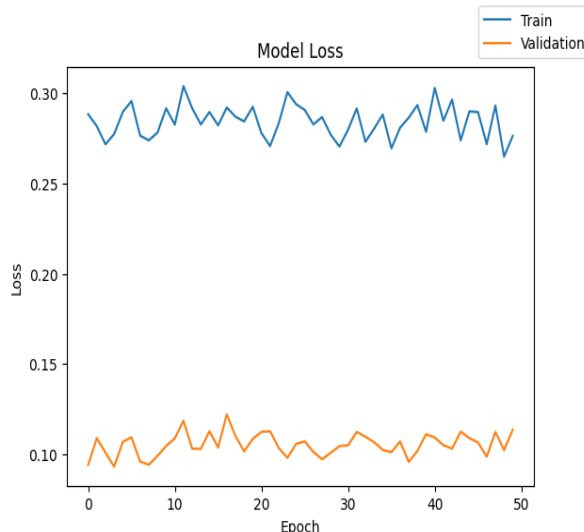
Training

- Epochs: 50
- Batch Size: 32
- Activation Function Hidden Layer 1: tanh
- Dropout (.50)
- Class Weight: 0: (1.05) 1: (18.02)
- Activation Function Hidden Layer 2: tanh
- Activation Function Hidden Layer 3: tanh
- Activation Function Output Layer: Sigmoid
- Optimizer: SGD
- Total Params: 1985
- Optimizer Metric: Recall

Validation

- Epochs: 50
- Batch Size: 32
- Activation Function Hidden Layer 1: tanh
- Dropout (.50)
- Class Weight: 0: (1.05) 1: (18.02)
- Activation Function Hidden Layer 2: tanh
- Activation Function Hidden Layer 3: tanh
- Activation Function Output Layer: Sigmoid
- Optimizer: SGD
- Total Params: 1985
- Optimizer Metric: Recall

Model Loss Graph



Training Time: 65.09

Performance Metrics

- Accuracy: 0.32
 - Recall: 0.43
 - Precision: 0.48
 - F1: 0.27
-
- Accuracy: 0.31
 - Recall: 0.43
 - Precision: 0.48
 - F1: 0.27

- **Model 6** is trained using 50 epochs and a batch size of 32. The neural network consists of three hidden layers with tanh activation functions and a Sigmoid activation in the output layer for binary classification. A dropout rate of 0.5 is applied to reduce overfitting. The model uses the SGD optimizer, and class weights are applied to address class imbalance and give more importance to the minority class (generator failures). The model is optimized using recall as the primary metric, which aligns with the objective of detecting generator failures early to reduce replacement costs.
- The recall score of 0.43 indicates a high number of false negatives with the model. In addition, the loss graph indicates significant underfitting. Model 6 has the worst performing levels when compared to previous models.

Final Model Selection

Training set performance comparison:

	Model 0	Model 1	Model 2	Best Performing Model Model 3	Model 4	Model 5	Model 6
Accuracy	0.987733	0.987667	0.986800	0.990800	0.993800	0.993867	0.320200
Recall	0.905262	0.901267	0.884404	0.956099	0.952030	0.947540	0.438195
Precision	0.974658	0.978385	0.988668	0.956099	0.988098	0.993614	0.484924
F1 Score	0.936872	0.936016	0.929547	0.956099	0.969282	0.969304	0.272178

Validation set performance comparison:

	Model 0	Model 1	Model 2	Model 3	Model 4	Model 5	Model 6
Accuracy	0.987000	0.987200	0.988400	0.991400	0.991200	0.991400	0.319600
Recall	0.905098	0.906897	0.899069	0.944667	0.932712	0.929433	0.436650
Precision	0.967342	0.967560	0.989673	0.971931	0.981994	0.987670	0.484504
F1 Score	0.933710	0.934849	0.939178	0.957839	0.955853	0.956465	0.271722

Model Objective

High Recall = Fewer False Negatives

Model Objective is to identify turbine generator failures (Actual Positive Cases)

High Recall = Fewer False Negatives

Model Objective is to identify turbine generator failures (Actual Positive Cases)

Model 3 is the best-performing model, achieving the highest recall (.956) while maintaining strong overall performance, making it the most effective model for detecting generator failures.

Test Performance of Best Model: Model 3

Neural Network Model Architecture

- ❑ Input Layer 1 – Activation Function: relu
- ❑ Dropout (.50)
- ❑ Hidden Layer: relu
- ❑ Hidden Layer: relu
- ❑ Output Layer: sigmoid
- ❑ Optimizer: SGD
- ❑ Optimizer Metric: Recall
- ❑ Class weight: cw_dict
- ❑ Epochs: 50
- ❑ Batch Size: 32

Neural Network Model Train & Validation Performance

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 64)	2,624
dropout_1 (Dropout)	(None, 64)	0
dense_5 (Dense)	(None, 32)	2,080
dense_6 (Dense)	(None, 16)	528
dense_7 (Dense)	(None, 1)	17

Total params: 5,249 (20.50 KB)
Trainable params: 5,249 (20.50 KB)
Non-trainable params: 0 (0.00 B)

		Accuracy	Recall	Precision	F1 Score
Train	0	0.9908	0.956099	0.956099	0.956099
Validation	0	0.9914	0.944667	0.971931	0.957839

Neural Network Model Test Set Performance

157/157 ————— 0s 1ms/step

	Accuracy	Recall	Precision	F1 Score	
Test	0	0.988	0.933627	0.95175	0.942477

- The model demonstrates very strong predictive performance on the unseen data.
- A recall of 93.36% means that the model would correctly identify 93.36% of the actual generator failures in the dataset.
- Model 3 is highly suitable for deployment.

Conclusions and Recommendations

The analysis demonstrates that neural network models can effectively predict wind turbine generator failures using historical sensor data. Multiple models were developed and evaluated, with the best-performing model (model 3) achieving a high recall percentage (95%), indicating strong capability in identifying potential failures before they occur.

The final model provides a reliable predictive tool that can support proactive maintenance decisions by identifying turbines at risk of failure. By implementing this solution, **ReneWind** can reduce unexpected downtime, lower maintenance and replacement costs, and improve the overall reliability and efficiency of wind turbine operations.

Recommendations:

- **Model Deployment:** Implement the model in the turbine monitoring system to flag generators with high failure risk.
- **Retraining:** Retrain the model periodically to improve prediction and adapt to changing operational conditions.
- **Expand Sensor Data Collection:** Collect additional operational data to further enhance prediction performance.
- **Decision Support:** Use the model as an information tool to assist maintenance teams to validate alerts before taking action.



Happy Learning !

